3.11 EEPROM MEMORY

MIKROELEKTRONIKA

The PIC16F887 microcontroller has 256 locations of data EEPROM controlled by the bits of the following registers:

- EECON1 (control register);
- EECON2 (control register);
- EEDAT (saves data ready for write and read); and
- EEADR (saves address of EEPROM location to be accessed).

In addition, EECON2 is not true register, it does not physically exist. It is used in data write program sequence only.

The EEDATH and EEADRH registers are used during EEPROM write and read. Both of them are also used for program (FLASH) memory write and read.

Since this is considered a risk zone (you surely do not want your microcontroller to accidentally delete it's own program), we will not discuss it further, but advise you to be careful.

EECON1 Register

	R/W (x)				R/W (x)	R/W (0)	R/S (0)	R/S (0)	Features
EECON1	EEPGD	-	-	-	WRERR	WREN	WR	RD	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

Legend

-	Bit is unimplemented
R/W	Readable/Writable bit
R	Readable bit
S	Bit can only be set
(0)	After reset, bit is cleared
(x)	After reset, bit is unknown

EEPGD – Program/Data EEPROM Select bit

- 1 – Access program memory.
- 0 – Access EEPROM memory.

WRERR – EEPROM Error Flag bit

- 1 – Write operation is prematurely terminated and error has occurred.
- 0 – Write operation completed.

WREN – EEPROM Write Enable bit.

- 1 – Write to data EEPROM is enabled.
- 0 – Write to data EEPROM is disabled.

WR – Write Control bit

- 1 – Initiates write to data EEPROM.
- 0 – Write to data EEPROM is complete.

RD – Read Control bit

- 1 – Initiates read from data EEPROM.
- 0 – Read from data EEPROM is disabled.

READ FROM EEPROM MEMORY

In order to read data EEPROM memory, follow the procedure below:

- **Step 1**: Write the address (00h – FFh) to the EEADR register.
- **Step 2**: Select EEPROM memory block by clearing the EEPGD bit of the EECON1 register.
- **Step 3**: To read location, set the RD bit of the same register.
- **Step 4**: Data is stored in the EEDAT register and is ready for use.

The following example illustrates the above procedure when writing a program in assembly language:

```
1 BSF STATUS,RP1   ;
2 BCF STATUS,RP0   ; Access bank 2
3 MOVF ADDRESS,W   ; Move address to the W register
4 MOVWF EEADR      ; Write address
5 BSF STATUS,RP0   ; Access bank 3
6 BCF EECON1,EEPGD ; Select EEPROM
7 BSF EECON1,RD    ; Read data
8 BCF STATUS,RP0   ; Access bank 2
9 MOVF EEDATA,W    ; Data is stored in the W register
```

The same program sequence written in C language looks as follows:

```
1 W = EEPROM_Read(ADDRESS);
```

The advantages of C language becomes more obvious, don't they?

WRITE DATA TO EEPROM MEMORY

Prior to writing data to EEPROM memory it is necessary to write the address to the EEADR register and data to the EEDAT register. All that's left is to follow a special sequence to initiate write for each byte. Interrupts must be disabled as long as this procedure is in progress.

The example below illustrates the above procedure when writing a program in assembly language:

```
 1 BSF STATUS,RP1
 2 BSF STATUS,RP0
 3 BTFSC EECON,WR1  ; Wait for the previous write to complete
 4 GOTO $-1         ;
 5 BCF STATUS,RP0   ; Bank 2
 6 MOVF ADDRESS,W   ; Move address to W
 7 MOVWF EEADR      ; Write address
 8 MOVF DATA,W      ; Move data to W
 9 MOVWF EEDATA     ; Write data
10 BSF STATUS,RP0   ; Bank 3
11 BCF EECON1,EEPGD ; Select EEPROM
12 BSF EECON1,WREN  ; Write to EEPROM enabled
13 BCF INCON,GIE    ; All interrupts disabled
14
15 ;Required Sentence
16 MOVLW 55h
17 MOVWF EECON2
18 MOVLW AAh
19 MOVWF EECON2
20 BSF EECON1,WR
21
22 BSF INTCON,GIE   ; Interrupts enabled
23 BCF EECON1,WREN  ; Write to EEPROM disabled
```

The same program sequence written in C language looks as follows:

```
1 W = EEPROM_Write(ADDRESS, W);
```

Need a comment?

Let's do it in mikroC…

```
// This example demonstrates the use of EEPROM Library in mikroC PRO for PIC.

char ii;       // Loop variable

void main(){
    ANSEL = 0; // Configure AN pins as digital I/O
    ANSELH = 0;
    PORTB = 0;
    PORTC = 0;
    PORTD = 0;
    TRISB = 0;
    TRISC = 0;
    TRISD = 0;

    for(ii = 0; ii &lt; 32; ii++)     // Fill data buffer
        EEPROM_Write(0x80+ii, ii); // Write data to address 0x80+ii

    EEPROM_Write(0x02,0xAA); // Write some data to EEPROM address 2
    EEPROM_Write(0x50,0x55); // Write some data to EEPROM address 0x50

    Delay_ms(1000);          // Blink PORTB and PORTC diodes
    PORTB = 0xFF;            // to indicate start of reading
    PORTC = 0xFF;
    Delay_ms(1000);

    PORTB = 0x00;
    PORTC = 0x00;
    Delay_ms(1000);

    PORTB = EEPROM_Read(0x02); // Read data from EEPROM address 2 and display it on PORTB
    PORTC = EEPROM_Read(0x50); // Read data from EEPROM address 0x50 and display it on PORTC
    Delay_ms(1000);

    for(ii = 0; ii &lt; 32; ii++) {      // Read 32 bytes block from address 0x80
        PORTD = EEPROM_Read(0x80+ii); // and display data on PORTD
        Delay_ms(250);
    }
}
```

At first glance, it is sufficient to turn the power on to make the microcontroller operate. At first glance, it is sufficient to turn the power off to make it stop operating. Only at first glance… In reality, start and end of operation are critical phases of which a special signal called RESET takes care.
